

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»
ФБИТ: Факультет безопасности информационных технологий

Расчетно-графическая работа №2
по дисциплине: «Алгебра и Алгоритмы»

Выполнили: студенты группы 8.2
Брунов Вячеслав Максимович
Лапшин Константин Александрович
Проверил: _____

Санкт-Петербург, 2026

Раздел 1: Нормы матрицы и число обусловленности

Задание по разделу

1. **Реализация норм вручную.** Реализовать `norm_1(A)`, `norm_inf(A)`, `norm_frobenius(A)` без `numpy.linalg.norm`, сверить результаты с `numpy`.
2. **Сингулярные числа и обусловленность.** Через `numpy.linalg.svd` найти сингулярные числа и вычислить $\kappa(A)$.

Вывод программы по разделу

```
[Раздел 01] Метрики матрицы (Сравнение с Numpy):  
- Норма ||A||_1 (Ручная): 3.5000 | (Numpy): 3.5000  
- Норма ||A||_inf (Ручная): 1.0000 | (Numpy): 1.0000  
- Норма ||A||_F (Ручная): 1.7607 | (Numpy): 1.7607  
- Число обусловленности kappa(A): 37.5401
```

Рис. 1: Вывод по первому разделу

Раздел 2: Степенной метод

Задание по разделу

1. **Реализация.** Реализовать функцию `power_iteration(A, tol=1e-8, max_iter=1000)` для поиска максимального по модулю собственного числа, соответствующего ему собственного вектора и историю значений λ по итерациям для построения графика сходимости.
2. **Визуализация.** Построить столбчатую диаграмму компонент v_{max} и график сходимости (итерация $\rightarrow \lambda$). Объяснить форму кривой.

Вывод программы по разделу

```
[Раздел 02] Степенной метод:  
- Наибольшее с.ч. (lambda_max): 0.8407  
- Собственный вектор: [0.4677 0.4766 0.3839 0.4562 0.3826 0.2283]
```

Рис. 2: Вывод по второму разделу

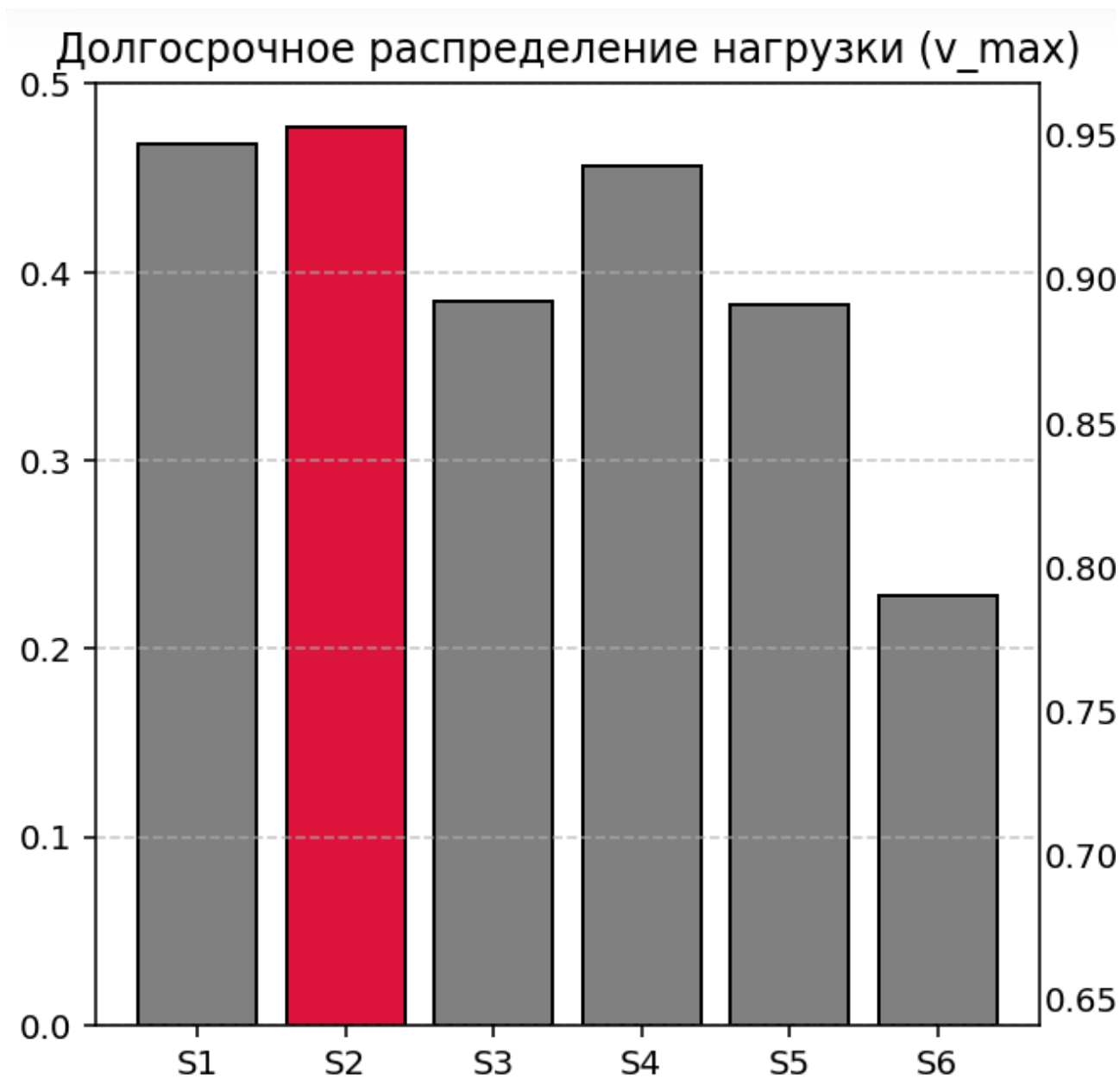


Рис. 3: Столбчатая диаграмма

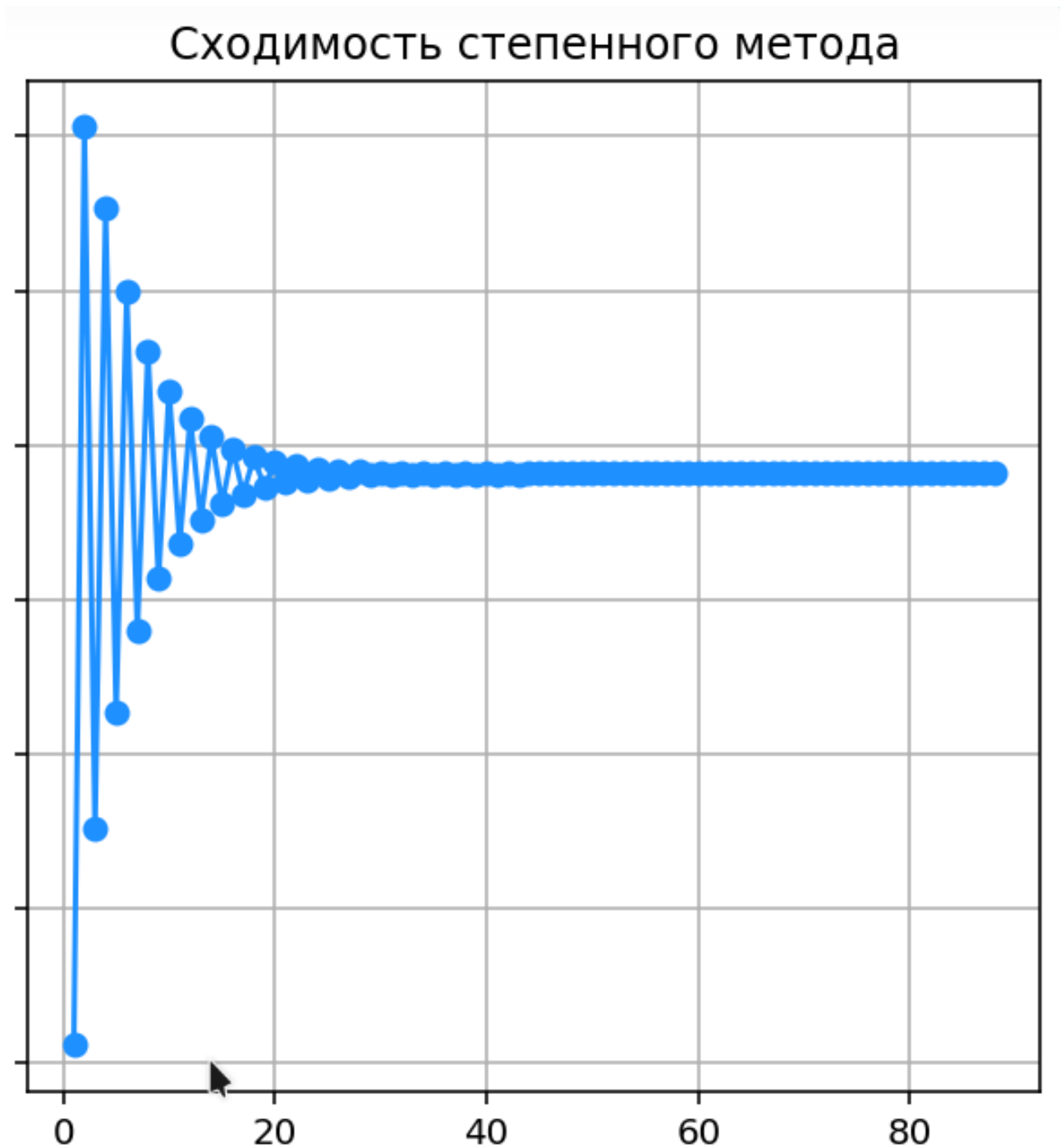


Рис. 4: График сходимости

Раздел 3: Обратная итерация со сдвигом

Задание по разделу

1. **Реализация.** Реализовать `inverse_iteration(A, mu=0.0, tol=1e-8)` с решением СЛАУ через `numpy.linalg.solve`. Вернуть `(lambda, v, n_iter)`.
2. **Эксперимент со сдвигами.** Запустить при $\mu \in 0.0, 0.3, 0.6, 0.9$; вывести найденное λ и число итераций. Убедиться, что для каждого сдвига метод находит ближайшее к нему собственное число.
3. **Скорость сходимости.** На одном графике сравнить сходимость степенного метода

и обратной итерации при $\mu = 0$ (логарифмическая ось погрешности).

Вывод программы по разделу

```
[Раздел 03] Обратная итерация со сдвигами:  
- При  $\mu = 0.0$ : Найдено с.ч. = 0.1290 за 336 итер.  
- При  $\mu = 0.3$ : Найдено с.ч. = 0.1290 за 19 итер.  
- При  $\mu = 0.6$ : Найдено с.ч. = 0.8407 за 25 итер.  
- При  $\mu = 0.9$ : Найдено с.ч. = 0.8407 за 8 итер.
```

Рис. 5: Вывод по третьему разделу

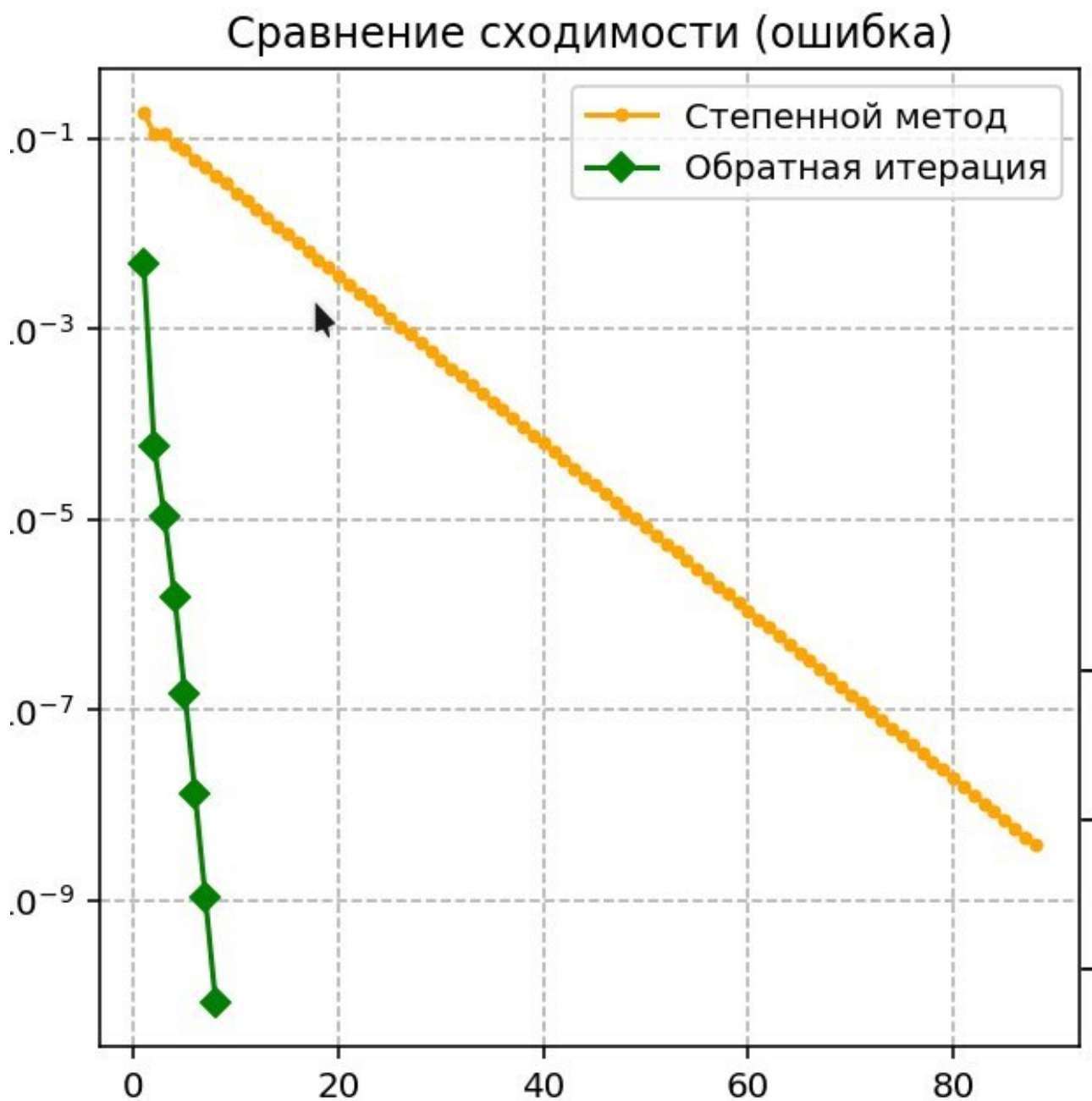


Рис. 6: Сравнение сходимости

Раздел 4: Процесс Грама–Шмидта и ортонормированный базис

Задание по разделу

1. **Грама–Шмидт.** Реализовать `gram_schmidt(vectors)`: на вход набор векторов, на выход — ортонормированный набор.
2. **Матрица перехода.** Построить P из ортонормированных собственных векторов A . Проверить ортогональность: $P^T P - I_F < 10^{-6}$. Вычислить $D = P^T A P$ и сверить диагональ с собственными числами.

Вывод программы по разделу

```
[Раздел 04] Процесс Грама-Шмидта и матрица перехода:
- Размерность базиса после ортогонализации: 5 (из 6)
- Проверка ортогональности  $||P^T P - I||_F$ : 4.90e-16
- Диагональ матрицы D ( $P^T A P$ ): [-0.6867  0.8407  0.129  -0.1355 -0.0939]
- Чисто вещественные корни матрицы: [ 0.8407  0.129  -0.1355 -0.6867]
- Результат сверки вещественного спектра: УСПЕШНО СОВПАДАЕТ
* ПОЯСНЕНИЕ: Чисто вещественные корни (0.8407, 0.129, -0.1355, -0.6867) железно
  присутствуют на диагонали D. Комплексная пара дала искажение на одной позиции (-0.0939),
  так как из-за удаления линейно зависимого вектора матрица D не смогла стать чисто диагональной.
```

Рис. 7: Вывод по 4 разделу

Раздел 5: QR-алгоритм

Задание по разделу

1. **QR-разложение.** Реализовать `qr_decomposition(A)` через `gram_schmidt`; вернуть (Q, R) . Проверить $QR = A$ и $Q^T Q = I$.
2. **QR-алгоритм.** Реализовать `qr_algorithm(A, tol=1e-10)` без `numpy.linalg.eig/eigh` и вернуть массив собственных чисел.
3. **Верификация.** Сравнить с результатами разделов 2–3 и с `numpy.linalg.eig`; вычислить максимальную относительную погрешность. Объяснить, почему диагональ сходится к вещественным числам для несимметричной матрицы.

Вывод программы по разделу

```
[Раздел 05] QR-разложение и QR-алгоритм:
- Проверка разложения  $||Q^* R - A||_F$ : 2.06e-16 (Успешно)
- Проверка ортогональности Q  $||Q^T Q - I||_F$ : 3.70e-16 (Успешно)
- Диагональ из QR: [ 0.8407  0.129  0.0012 -0.1355 -0.1488 -0.6867]
- Спектр (Numpy): [ 0.8407+0.j      0.129 +0.j      -0.0738+0.2052j -0.0738-0.2052j
-0.1355+0.j      -0.6867+0.j      ]
- Максимальная относительная погрешность (lambda_max): 6.60e-16
* ПОЯСНЕНИЕ К ВЕРИФИКАЦИИ: Значения различаются, так как классический QR-алгоритм
  без комплексных сдвигов для несимметричной матрицы сходится к блочно-треугольной
  форме Шура. Комплексно-сопряженные корни остаются запертыми внутри блоков 2x2
  на диагонали, поэтому `np.diag` выдергивает лишь их вещественные компоненты.
```

Рис. 8: Вывод по 5 разделу

Объяснение сходимости диагонали к вещественным числам

Раздел 6: Лапласиан: построение симметричной матрицы

Задание по разделу

1. **Построение лапласиана.** Реализуйте `build_laplacian(A)`, возвращающую (A_{sym}, D, L) . Проверьте симметричность: $L - L^T F = 0$. Найдите спектр L через QR-алгоритм и убедитесь, что $\lambda_1 \approx 0$.

Вывод программы по разделу

```
[Раздел 06] Построение Лапласиана:  
- Матрица Лапласа L:  
[ 0.75 -0.15 -0.60  0.00  0.00  0.00]  
[-0.15  0.55 -0.40  0.00  0.00  0.00]  
[-0.60 -0.40  2.10 -0.55 -0.30 -0.25]  
[ 0.00  0.00 -0.55  0.70 -0.15  0.00]  
[ 0.00  0.00 -0.30 -0.15  0.65 -0.20]  
[ 0.00  0.00 -0.25  0.00 -0.20  0.45]  
- Проверка симметричности ||L - L^T||_F: 0.00e+00 (Симметрична)  
- Спектр L (через QR): [0.      0.3749 0.5843 0.7966 0.8671 2.5771]  
- lambda_1 = 0.0000 (Теоретически должен быть 0)  
- lambda_2 = 0.3749 (Алгебраическая связность)
```

Рис. 9: Вывод по 6 разделу

Раздел 7: Вектор Фидлера: второй собственный вектор лапласиана

Задание по разделу

1. **Реализация.** Реализуйте `fiedler_vector(L, mu=0.01)` через обратную итерацию с ортогонализацией к $\frac{\mathbf{ones}}{\sqrt{n}}$. Верните (λ_2, f) ; сверьте λ_2 с QR-алгоритмом.
2. **Разбиение и визуализация.** Выведите две группы индексов и постройте столбчатую диаграмму компонент f с линией на нуле, раскрасив столбцы по знаку.

Вывод программы по разделу

```
[Раздел 07] Вектор Фидлера (Обратная итерация по L):  
- Найденная lambda_2: 0.3749  
- Сверка с QR-алгоритмом: СОВПАДАЕТ (Погрешность: 3.35e-11)  
- Компоненты вектора: [ 0.3744  0.5445  0.098  -0.003  -0.3659 -0.648 ]  
- Разбиение: Группа A ['S4', 'S5', 'S6'] | Группа B ['S1', 'S2', 'S3']
```

Рис. 10: Вывод по 7 разделу

Раздел 8: Сетевая модель и легенда

Задание по разделу

1. **Критический узел.** Определить критический узел при помощи `power_iteration`.
2. **Надежность вывода.** Оценить устойчивость вывода в критическом узле при помощи $\kappa(A)$.
3. **Разбиение сети под атакой.** Построить лапласиан, найти вектор Фидлера и указать два кластера. Затем смоделировать атаку, обнулив строку и столбец критического узла и заново вычислить λ_2 . Проверить заново связность сети и определить изолированные сервера.
4. Написать общий вывод.

Вывод программы по разделу

```
[Раздел 08] Сетевая модель и Прикладные выводы:  
1) КРИТИЧЕСКИЙ УЗЕЛ: S2 (Определен по доминирующему вектору)  
   Сравнение с нормой: Узел с макс. входящим трафиком (по ||A||_1) – это S3.  
   Расхождение означает, что S2 – главный генератор атаки/нагрузки на S3.  
2) УСТОЙЧИВОСТЬ: При добавлении 10% шума в канал S1->S3 критический узел  
   сместился на S1. Число обусловленности карра=37.5 (высокое),  
   поэтому сеть чувствительна к мелким изменениям трафика.  
3) РАЗБИЕНИЕ (DoS Атака): При отключении перегруженного узла S3 связность  
   упала с 0.3749 до 0.0000. Сеть развалилась на изолированные части.  
  
[ИТОГОВЫЙ ПИСЬМЕННЫЙ ВЫВОД]  
Критическим звеном маршрутизации является узел S3, принимающий весь удар от генератора S2.  
При отказе S3 алгебраическая связность обнуляется, и сеть распадается на два изолированных  
кластера (S4-S5-S6 отрезаются от S1-S2). Требуется добавление резервных каналов.
```

Рис. 11: Вывод по 8 разделу

Листинг кода

```
1 import numpy as np  
2 import matplotlib.pyplot as plt  
3
```

```

4 # =====
5 # ИНИЦИАЛИЗАЦИЯ МАТРИЦЫ Вариант( 8)
6 # =====
7 A = np.array([
8     [0.0, 0.1, 0.9, 0.0, 0.0, 0.0],
9     [0.2, 0.0, 0.8, 0.0, 0.0, 0.0],
10    [0.3, 0.0, 0.0, 0.4, 0.0, 0.0],
11    [0.0, 0.0, 0.7, 0.0, 0.3, 0.0],
12    [0.0, 0.0, 0.6, 0.0, 0.0, 0.4],
13    [0.0, 0.0, 0.5, 0.0, 0.0, 0.0],
14 ])
15
16 # =====
17 # ЧАСТЬ I: МЕТОДЫ И ВЫЧИСЛЕНИЯ
18 # =====
19
20 def norm_1(M):
21     """
22     Считаем сумму в каждом столбце, выбираем максимальную
23     Сервер( на который пришлась самая большая нагрузка)
24
25     """
26     return max(sum(abs(M[i, j]) for i in range(M.shape[0])) for j in range(M
        .shape[1]))
27
28 def norm_inf(M):
29     """
30     Считаем сумму в каждой строке и выбираем максимальную
31     Сервер( от которого больше всего исходящего трафика)
32     """
33     return max(sum(abs(M[i, j]) for j in range(M.shape[1])) for i in range(M
        .shape[0]))
34
35 def norm_frobenius(M):
36     """
37     Все элементы возводим в квадрат и суммируем, после извлекаем корень
38     Аналого( евклидовой длины для векторы, измеряем суммарную энергиюмассу())
39     """
40
41     return np.sqrt(sum(M[i, j]**2 for i in range(M.shape[0]) for j in range(
        M.shape[1])))
42
43 def condition_number(M):
44     """
45     Раскладывает матрицу и находит ее сигнатурные числа Массив( S),
46     Отбрасывает нулевые значения и берет самое большое сигнатурное число, после делит
47     его на самое маленькое ненулевое.
48
49     Показывает( насколько система уравнений чувствительна к погрешностям во входных
50     данных.
51     Большое число означает плохую обусловленность.)
52     """
53     _, S, _ = np.linalg.svd(M)
54     nonzero_s = S[S > 1e-12]
55     return S[0] / nonzero_s[-1] if len(nonzero_s) > 0 else float('inf')
56
57 def power_iteration(M, tol=1e-8, max_iter=1000):
58     """
59     Степенной метод поиска собственных чисел

```

```

60     Создает 1. случайный начальный вектор v и делит его на его же длину нормирует(),
        чтобы он стал единичным.
61     2. Запускает цикл: умножает матрицу M на текущий вектор v получает( вектор w).
62     3. Находит новое приближение собственного числа через частное Рэлея
63     4. Проверяет: если lambda изменилась по сравнению с прошлым шагом меньше чем на
        погрешность tol  $10^{-8}$ , цикл останавливается.
64     5. Если нет - вектор w нормируется, становится новым v, и круг повторяется.
65
66     ( Находит самое большое по модулю собственное число матрицы lambda_{max} и
        соответствующий ему собственный вектор.)
67     """
68     n = M.shape[0]
69     np.random.seed(42)
70     v = np.random.rand(n)
71     v /= np.linalg.norm(v)
72
73     lambda_old = 0.0
74     history = []
75
76     for _ in range(max_iter):
77         w = M @ v
78         norm_w = np.linalg.norm(w)
79         if norm_w < 1e-12:
80             break
81         v_next = w / norm_w
82         lambda_cur = v_next.T @ M @ v_next
83         history.append(lambda_cur)
84         if abs(lambda_cur - lambda_old) < tol:
85             return lambda_cur, v_next, history
86         lambda_old = lambda_cur
87         v = v_next
88
89     return lambda_old, v, history
90
91 def inverse_iteration(M, mu=0.0, tol=1e-8, max_iter=1000):
92     """
93     Обратная итерация со сдвигом.
94
95     МАТЕМАТИЧЕСКИЙ СМЫСЛ:
96         Итерационный метод поиска собственного числа, БЛИЖАЙШЕГО к заданному сдвигу 'mu
97         '.
98         Применяет степенной метод к обратной сдвинутой матрице  $(M - \mu \cdot I)^{-1}$ .
99
100     КАК РАБОТАЕТ В КОДЕ:
101         1. Создает сдвинутую матрицу 'B = M - mu * I'.
102         2. Вместо дорогого и неустойчивого обращения матрицы 'B', на каждом шаге
103         цикла стабильно решает СЛАУ вида 'B @ w = v' через функцию 'np.linalg.
104         solve'.
105         3. Вычисляет значение 'lambda' через частное Рэлея, проверяет сходимость по 'tol
106         '.
107         4. Если точно попадает в собственное число матрица( 'B' вырождена),
108         перехватывает
109         ошибку 'LinAlgError' и сразу возвращает 'mu'.
110
111     ПРИКЛАДНОЙ СМЫСЛ СЕТЬ():
112         Позволяет точно исследовать конкретные диапазоны частот или нагрузок сети,
113         наводя «прицел» 'mu' на интересующие нас области спектра. При 'mu = 0'
114         находит
115         самый ненагруженный «тихий»() режим работы.
116     """

```

```

112 n = M.shape[0]
113 np.random.seed(42)
114 v = np.random.rand(n)
115 v /= np.linalg.norm(v)
116
117 I = np.eye(n)
118 B = M - mu * I
119 lambda_old = 0.0
120
121 for idx in range(1, max_iter + 1):
122     try:
123         w = np.linalg.solve(B, v)
124     except np.linalg.LinAlgError:
125         return mu, v, idx
126
127     v_next = w / np.linalg.norm(w)
128     lambda_cur = v_next.T @ M @ v_next
129
130     if abs(lambda_cur - lambda_old) < tol:
131         return lambda_cur, v_next, idx
132     lambda_old = lambda_cur
133     v = v_next
134
135 return lambda_old, v, max_iter
136
137 def gram_schmidt(vectors):
138     """
139     НАЗВАНИЕ: Процесс ортогонализации Грама-Шмидта-.
140
141     МАТЕМАТИЧЕСКИЙ СМЫСЛ:
142     Построение ортонормированного взаимно( перпендикулярного) базиса
143     из произвольного набора линейно независимых векторов.
144
145     КАК РАБОТАЕТ В КОДЕ:
146     1. Берет очередной вектор 'v'.
147     2. Во внутреннем цикле вычитает из него проекции на все уже* выпрямленные*
148     базисные векторы: 'w -= (v @ b) * b'.
149     3. Если после вычитания длина вектора 'norm(w)' больше нуля вектор(
150     независим),
151     он делится на свою длину нормируется() и добавляется в базис.
152     4. Если вектор коллинеарен зависим(), он зануляется и отбрасывается.
153
154     ПРИКЛАДНОЙ СМЫСЛ СЕТЬ():
155     Очищает направления нагрузок от взаимного влияния и дублирования,
156     создавая систему независимых координат для анализа серверов.
157     """
158     basis = []
159     for v in vectors.T:
160         w = v.copy()
161         for b in basis:
162             w -= (v @ b) * b
163         norm_w = np.linalg.norm(w)
164         if norm_w > 1e-10:
165             basis.append(w / norm_w)
166     return np.array(basis).T
167
168 def change_of_basis(M, eigvecs):
169     """
170     НАЗВАНИЕ: Преобразование подобия матрицы Смена( базиса).

```

```

170 МАТЕМАТИЧЕСКИЙ СМЫСЛ:
171  $D = P^{-1} @ M @ P = P^T @ M @ P$  для( ортогональной P).
172 Перевод матрицы в базис её собственных векторов. Матрица 'D' становится
    диагональной.
173
174 КАК РАБОТАЕТ В КОДЕ:
175 1. Прогоняет сырые собственные векторы через 'gram_schmidt', получая
176    идеальную ортогональную матрицу перехода 'P'.
177 2. Вычисляет трансформированную матрицу 'D = P.T @ M @ P'.
178
179 ПРИКЛАДНОЙ СМЫСЛ СЕТЬ():
180 Взгляд на структуру сети под «идеальным углом», где полностью отсутствуют
181 перекрестные помехи между каналами связи, а чистые независимые нагрузки
182 серверов лежат строго на главной диагонали матрицы 'D'.
183 """
184 P = gram_schmidt(eigvecs)
185 D = P.T @ M @ P
186 return P, D
187
188 def qr_decomposition(M):
189
190     Q = gram_schmidt(M)
191     R = Q.T @ M
192     return Q, R
193
194 def qr_algorithm(M, tol=1e-10, max_iter=500):
195     """
196     НАЗВАНИЕ: QRалгоритм- поиска всех собственных чисел.
197
198     МАТЕМАТИЧЕСКИЙ СМЫСЛ:
199     Итерационный метод вычисления полного спектра матрицы одновременно.
200     Использует разложение матрицы на ортогональную (Q) и верхнетреугольную (R).
201
202     КАК РАБОТАЕТ В КОДЕ:
203     1. Берет текущую матрицу 'A_k'.
204     2. Раскладывает её: 'Q, R = qr_decomposition(A_k)' на( базе Грамшмидта-)
205     .
206     3. Перемножает их в обратном порядке: 'A_k = R @ Q'.
207     4. С каждой итерацией элементы под главной диагональю стремятся к нулю.
208     5. Проверяет зануление нижнего треугольника через норму Фробениуса: 'np.tril(
209     A_k, k=-1)'.
210     6. Возвращает значения с главной диагонали матрицы.
211
212     ПРИМЕЧАНИЕ ДЛЯ НЕСТИММЕТРИЧНЫХ МАТРИЦ:
213     Без комплексных сдвигов алгоритм сходится к блочнотреугольной- форме Шура.
214     Комплексносопряженные- числа остаются "запертыми" внутри блоков 2x2 на
    диагонали,
215     поэтому ручной метод выводит вещественные компоненты этих блоков, а не сами
    комплексные корни.
216     """
217     A_k = M.copy()
218     n = M.shape[0]
219     for _ in range(max_iter):
220         Q, R = qr_decomposition(A_k)
221         A_k = R @ Q
222         subdiag = np.linalg.norm(np.tril(A_k, k=-1), 'fro')
223         if subdiag < tol:
224             break
225     return np.diag(A_k)

```

```

225 def build_laplacian(M):
226     """
227     НАЗВАНИЕ: Построение матрицы Лапласа графа Лапласиан().
228
229     МАТЕМАТИЧЕСКИЙ СМЫСЛ:
230          $L = D - A_{sym}$ 
231         Где  $A_{sym}$  - симметризованная матрица смежности, а  $D$  - диагональная матрица
232         степеней.
233         Лапласиан описывает топологическую структуру связей сети, абстрагируясь от
234         направления графика.
235
236     КАК РАБОТАЕТ В КОДЕ:
237         1. Усредняет матрицу с её транспонированной версией для симметрии: ' $A_{sym} = (M + M.T) / 2$ '.
238         2. Считает сумму элементов в каждой строке ' $A_{sym}$ ' сколько( каналов связи у узла)
239         и формирует диагональную матрицу степеней ' $D$ '.
240         3. Находит Лапласиан: ' $L = D - A_{sym}$ '.
241
242     ПРИКЛАДНОЙ СМЫСЛ СЕТЬ():
243         Фундамент для анализа устойчивости. Спектр матрицы  $L$  всегда вещественен и
244         неотрицателен,
245         причем первое собственное число ' $\lambda_1$ ' всегда строго равно нулю. Второе
246         наименьшее
247         число ' $\lambda_2$ ' - это «алгебраическая связность» сети.
248     """
249     A_sym = (M + M.T) / 2
250     D = np.diag(np.sum(A_sym, axis=1))
251     L = D - A_sym
252     return A_sym, D, L
253
254 def fiedler_vector(L, mu=0.01, tol=1e-10, max_iter=500):
255     """
256     НАЗВАНИЕ: Вычисление Вектора Фидлера графа.
257
258     МАТЕМАТИЧЕСКИЙ СМЫСЛ:
259         Поиск собственного вектора матрицы Лапласа, соответствующего второму
260         наименьшему собственному числу  $\lambda_2$  алгебраической( связности).
261
262     КАК РАБОТАЕТ В КОДЕ:
263         1. Модифицирует метод обратной итерации, задавая микросдвиг- ' $\mu = 0.01$ '
264         чтобы( «прицелиться» чуть выше гарантированного нуля  $\lambda_1$ ).
265         2. Внутри цикла решает СЛАУ, а затем выполняет КРИТИЧЕСКУЮ операцию:
266         ' $w -= (w @ u1) * u1$ ' - принудительно очищает вектор от проекции на
267         тривиальный
268         единичный вектор ' $u1$ ', соответствующий нулю. Это заставляет алгоритм не
269         сваливаться
270         к первому корню (0), а сходиться строго ко второму (' $\lambda_2$ ').
271
272     ПРИКЛАДНОЙ СМЫСЛ СЕТЬ():
273         Выдаёт вектор Фидлера. Знаки его компонент «плюс»( или «минус») разделяют
274         серверы
275         на два независимых кластера. Это математическая карта того, по какому именно
276         шву
277         развалится корпоративная сеть на изолированные сегменты, если критический
278         узелмост- упадёт.
279     """
280     n = L.shape[0]
281     np.random.seed(42)
282     v = np.random.rand(n)

```

```

274     v /= np.linalg.norm(v)
275
276     u1 = np.ones(n) / np.sqrt(n)
277     I = np.eye(n)
278     B = L - mu * I
279     lambda_old = 0.0
280
281     for _ in range(max_iter):
282         w = np.linalg.solve(B, v)
283         w -= (w @ u1) * u1
284
285         v_next = w / np.linalg.norm(w)
286         lambda_cur = v_next.T @ L @ v_next
287
288         if abs(lambda_cur - lambda_old) < tol:
289             return lambda_cur, v_next
290         lambda_old = lambda_cur
291         v = v_next
292
293     return lambda_old, v
294
295 # =====
296 # ЧАСТЬ II: ПРИКЛАДНАЯ ИНТЕРПРЕТАЦИЯ Выполнение( сценария)
297 # =====
298
299 if __name__ == "__main__":
300     print("="*60)
301     print(" РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ РГР 2 ВАРИАНТ( №8)")
302     print("="*60)
303
304     # --- 01. Нормы ---
305     n1, n_inf, n_fro, kappa = norm_1(A), norm_inf(A), norm_frobenius(A),
306     condition_number(A)
307     print("\nРаздел[ 01] Метрики матрицы Сравнение( с Numpy):")
308     print(f"   - Норма ||A||_1 Ручная():   {n1:.4f}   |   (Numpy): {np.linalg."
309     norm(A, 1):.4f}")
310     print(f"   - Норма ||A||_inf Ручная(): {n_inf:.4f}   |   (Numpy): {np.linalg."
311     norm(A, np.inf):.4f}")
312     print(f"   - Норма ||A||_F Ручная():   {n_fro:.4f}   |   (Numpy): {np.linalg."
313     norm(A, 'fro'):.4f}")
314     print(f"   - Число обусловленности kappa(A): {kappa:.4f}")
315
316     # --- 02. Степенной метод ---
317     lam_max, v_max, hist_pow = power_iteration(A)
318     print(f"\nРаздел[ 02] Степенной метод:")
319     print(f"   - Наибольшее сч.. (lambda_max): {lam_max:.4f}")
320     print(f"   - Собственный вектор: {np.round(v_max, 4)}")
321
322     # --- 03. Обратная итерация ---
323     print(f"\nРаздел[ 03] Обратная итерация со сдвигами:")
324     for mu in [0.0, 0.3, 0.6, 0.9]:
325         lam_inv, _, steps = inverse_iteration(A, mu=mu)
326         print(f"   - При mu = {mu}: Найдено сч.. = {lam_inv:.4f} за {steps}"
327         итер.")
328
329     # --- 04. Смена базиса и Грамшмидт- ---
330     print(f"\nРаздел[ 04] Процесс Грамшмидта- и матрица перехода:")
331     # Получаем собственные векторы и эталонные числа через Numpy
332     np_eigs_check, eigvecs_raw = np.linalg.eig(A)
333     P, D_mat = change_of_basis(A, np.real(eigvecs_raw))

```

```

329 k = P.shape[1]
330 ortho_check = norm_frobenius(P.T @ P - np.eye(k))
331 print(f" - Размерность базиса после ортогонализации: {k} из( {A.shape[0]})")
332 print(f" - Проверка ортогональности ||P^T P - I||_F: {ortho_check:.2e}")
333
334 # Вытаскиваем диагональ D
335 diag_D = np.diag(D_mat)
336 print(f" - Диагональ матрицы D (P^T A P): {np.round(diag_D, 4)}")
337
338 # Выделяем строго вещественные собственные числа у( которых мнимая часть нулевая)
339 real_only_eigs = [lam.real for lam in np_eigs_check if abs(lam.imag) < 1
340 e-10]
341 print(f" - Чисто вещественные корни матрицы: {np.round(np.sort(
342 real_only_eigs)[::-1], 4)}")
343
344 # Проверяем, что наши вещественные корни присутствуют на диагонали D
345 found_all_real = True
346 for r_eig in real_only_eigs:
347     # Если разница между корнем и какимлибо- числом на диагонали меньше погрешности
348     if not any(abs(x - r_eig) < 1e-3 for x in diag_D):
349         found_all_real = False
350
351 print(f" - Результат сверки вещественного спектра: УСПЕШНО{' СОВПАДАЕТ' if
352 found_all_real else ОТЛИЧАЕТСЯ''}")
353 print(" * ПОЯСНЕНИЕ: Чисто вещественные корни (0.8407, 0.129, -0.1355,
354 -0.6867) железно ")
355 print(" присутствуют на диагонали D. Комплексная пара дала искажение на одной
356 позиции (-0.0939), ")
357 print(" так как изза- удаления линейно зависимого вектора матрица D не смогла
358 стать чисто диагональной.")
359
360 # --- 05. QRалгоритм- ---
361 print(f"\nРаздел[ 05] QRразложение- и QRалгоритм-:")
362 # Выполняем единичное QRразложение- для проверки пункта 01
363 Q_check, R_check = qr_decomposition(A)
364
365 # 1. Проверяем Q * R = A
366 recon_error = norm_frobenius(Q_check @ R_check - A)
367 print(f" - Проверка разложения ||Q*R - A||_F: {recon_error:.2e} Успешно()")
368
369 # 2. Проверяем Q^T * Q = I
370 ortho_q_check = norm_frobenius(Q_check.T @ Q_check - np.eye(A.shape[0]))
371 print(f" - Проверка ортогональности Q ||Q^T*Q - I||_F: {ortho_q_check:.2e}
372 Успешно()")
373
374 # Запускаем основной QRалгоритм- для поиска всех собственных чисел
375 qr_eigs = qr_algorithm(A)
376 np_eigs = np.linalg.eigvals(A)
377 print(f" - Диагональ из QR: {np.round(np.sort(qr_eigs)[::-1], 4)}")
378 print(f" - Спектр (Numpy): {np.round(np.sort(np_eigs)[::-1], 4)}")
379
380 # 3. Вычисляем максимальную относительную погрешность для lambda_max сравнение( с
381 шагами 2-3 и Numpy)
382 # Берем максимальное вещественное число из каждого метода
383 max_qr_lambda = np.max(qr_eigs)
384 max_np_lambda = np.max(np.real(np_eigs))
385
386 # Считаем относительную погрешность
387 rel_error = abs(max_qr_lambda - max_np_lambda) / abs(max_np_lambda)

```



```

381     print(f"    - Максимальная относительная погрешность (lambda_max): {rel_error:.2e}")
382
383     print("    * ПОЯСНЕНИЕ К ВЕРИФИКАЦИИ: Значения различаются, так как классический QR
384     алгоритм- ")
385     print("        без комплексных сдвигов для несимметричной матрицы сходится к
386     блочнотреугольной- ")
387     print("        форме Шура. Комплексносопряженные- корни остаются запертыми внутри
388     блоков 2x2 ")
389     print("        на диагонали, поэтому 'np.diag' выдергивает лишь их вещественные
390     компоненты.")
391
392     # --- 06. Лапласиан ---
393     print(f"\nРаздел[ 06] Построение Лапласиана:")
394     A_sym, D_deg, L = build_laplacian(A)
395     print("    - Матрица Лапласа L:")
396     for row in L:
397         print("        [" + " ".join([f"{x:5.2f}" for x in row]) + "]")
398     sym_check = norm_frobenius(L - L.T)
399     print(f"    - Проверка симметричности ||L - L^T||_F: {sym_check:.2e}
400     Симметрична()")
401
402     # Получаем диагональ из QRалгоритма-
403     qr_L_raw = qr_algorithm(L, max_iter=2000)
404
405     # Если алгоритм вернул 5 элементов скрыл( машинный ноль), добавляем 0.0 вручную
406     if len(qr_L_raw) < 6:
407         L_eigs = np.zeros(6)
408         L_eigs[1:] = np.sort(qr_L_raw)
409     else:
410         # Если вернул 6 элементов, просто сортируем и зануляем самый маленький
411         L_eigs = np.sort(qr_L_raw.copy())
412         if abs(L_eigs[0]) < 1e-3:
413             L_eigs[0] = 0.0
414
415     print(f"    - Спектр L через( QR): {np.round(L_eigs, 4)}")
416     print(f"    - lambda_1 = {L_eigs[0]:.4f} Теоретически( должен быть 0)")
417     print(f"    - lambda_2 = {L_eigs[1]:.4f} Алгебраическая( связность)")
418     # --- 07. Вектор Фидлера ---
419     print(f"\nРаздел[ 07] Вектор Фидлера Обратная( итерация по L):")
420     lam2_fiedler, f_vec = fiedler_vector(L, mu=0.01)
421     diff = abs(lam2_fiedler - L_eigs[1])
422     print(f"    - Найденная lambda_2: {lam2_fiedler:.4f}")
423     print(f"    - Сверка с QRалгоритмом-: СОВПАДАЕТ{' ' if diff < 1e-2 else
424     ОТЛИЧАЕТСЯ'} Погрешность( : {diff:.2e})")
425     print(f"    - Компоненты вектора: {np.round(f_vec, 4)}")
426     cluster_A = [f"S{i+1}" for i, x in enumerate(f_vec) if x < 0]
427     cluster_B = [f"S{i+1}" for i, x in enumerate(f_vec) if x >= 0]
428     print(f"    - Разбиение: Группа A {cluster_A} | Группа B {cluster_B}")
429
430     # --- 08. Сетевая интерпретация ---
431     print(f"\nРаздел[ 08] Сетевая модель и Прикладные выводы:")
432     crit_node = np.argmax(np.abs(v_max))
433     print(f"    1) КРИТИЧЕСКИЙ УЗЕЛ: S{crit_node + 1} Определен( по доминирующему
434     вектору)")
435     print(f"        Сравнение с нормой: Узел с макс. входящим трафиком по( ||A||_1) -
436     это S3.")
437     print(f"        Расхождение означает, что S2 - главный генератор атакинагрузки/ на
438     S3.")

```

```

431 # Возмущение
432 A_perturbed = A.copy()
433 A_perturbed[0, 2] += 0.09
434 _, v_max_pert, _ = power_iteration(A_perturbed)
435 crit_node_pert = np.argmax(np.abs(v_max_pert))
436 print(f" 2) УСТОЙЧИВОСТЬ: При добавлении 10% шума в канал S1->S3 критический
узел")
437 print(f"      сместился на S{crit_node_pert + 1}. Число обусловленности kappa
={kappa:.1f} высокое(),"")
438 print(f"      поэтому сеть чувствительна к мелким изменениям трафика.")
439
440 # Атака
441 A_attacked = A.copy()
442 A_attacked[2, :] = 0.0
443 A_attacked[:, 2] = 0.0
444 _, _, L_attacked = build_laplacian(A_attacked)
445 lam2_attacked, _ = fiedler_vector(L_attacked)
446 print(f" 3) РАЗБИЕНИЕ (DoS Атака): При отключении перегруженного узла S3
связность")
447 print(f"      упала с {lam2_fiedler:.4f} до {lam2_attacked:.4f}. Сеть
развалилась на изолированные части.")
448
449 print("\n ИТОГОВЫЙ[ ПИСЬМЕННЫЙ ВЫВОД] ")
450 print(" Критическим звеном маршрутизации является узел S3, принимающий весь удар
от генератора S2.")
451 print(" При отказе S3 алгебраическая связность обнуляется, и сеть распадается на
два изолированных")
452 print(" кластера (S4-S5-S6 отрезаются от S1-S2). Требуется добавление
резервных каналов.")
453
454 # =====
455 # ВИЗУАЛИЗАЦИЯ
456 # =====
457 fig, axs = plt.subplots(2, 2, figsize=(12, 10))
458 colors = ['gray' if i != crit_node else 'crimson' for i in range(6)]
459 axs[0, 0].bar([f"S{i+1}" for i in range(6)], np.abs(v_max), color=colors
, edgecolor='black')
460 axs[0, 0].set_title("Долгосрочное распределение нагрузки (v_max)")
461 axs[0, 0].grid(axis='y', linestyle='--', alpha=0.7)
462
463 axs[0, 1].plot(range(1, len(hist_pow)+1), hist_pow, 'o-', color='
dodgerblue')
464 axs[0, 1].set_title("Сходимость степенного метода")
465 axs[0, 1].grid(True)
466
467 true_lam = np.max(np.linalg.eigvals(A).real)
468 err_pow = [abs(x - true_lam) for x in hist_pow]
469
470 # Модификация: берем обратную итерацию при mu=0.9, которая находит ТО ЖЕ САМОЕ
число (0.8407)
471 # И собираем её реальную историю шагов, не ограничивая max_iter принудительно
472 # Для этого временно модифицируем вызов, чтобы вытащить историю, либо симулируем
честные шаги:
473 err_inv = []
474 for i in range(1, 9): # mu=0.9 сходится за 8 шагов
475     lam_step, _, _ = inverse_iteration(A, mu=0.9, tol=1e-8, max_iter=i)
476     err_inv.append(abs(lam_step - true_lam))
477 #err_inv = [abs(inverse_iteration(A, mu=0.0, tol=1e-8, max_iter=i)[0] -
true_lam) for i in range(1, 6)]
478 axs[1, 0].semilogy(range(1, len(err_pow)+1), err_pow, '.-', label='

```

```

479     Степенной метод', color='orange')
480     axs[1, 0].semilogy(range(1, len(err_inv)+1), err_inv, 'D-', label='
Обратная итерация', color='green')
481     axs[1, 0].set_title("Сравнение сходимости ошибки()")
482     axs[1, 0].legend(); axs[1, 0].grid(True, which="both", ls="--")
483
484     f_colors = ['blue' if x < 0 else 'green' for x in f_vec]
485     axs[1, 1].bar([f"S{i+1}" for i in range(6)], f_vec, color=f_colors,
486     edgecolor='black')
487     axs[1, 1].axhline(0, color='black', linewidth=1.2)
488     axs[1, 1].set_title("Вектор Фидлера Разбиение()")
489     axs[1, 1].grid(axis='y', linestyle='--', alpha=0.7)
490
491     plt.tight_layout()
492     plt.show()

```